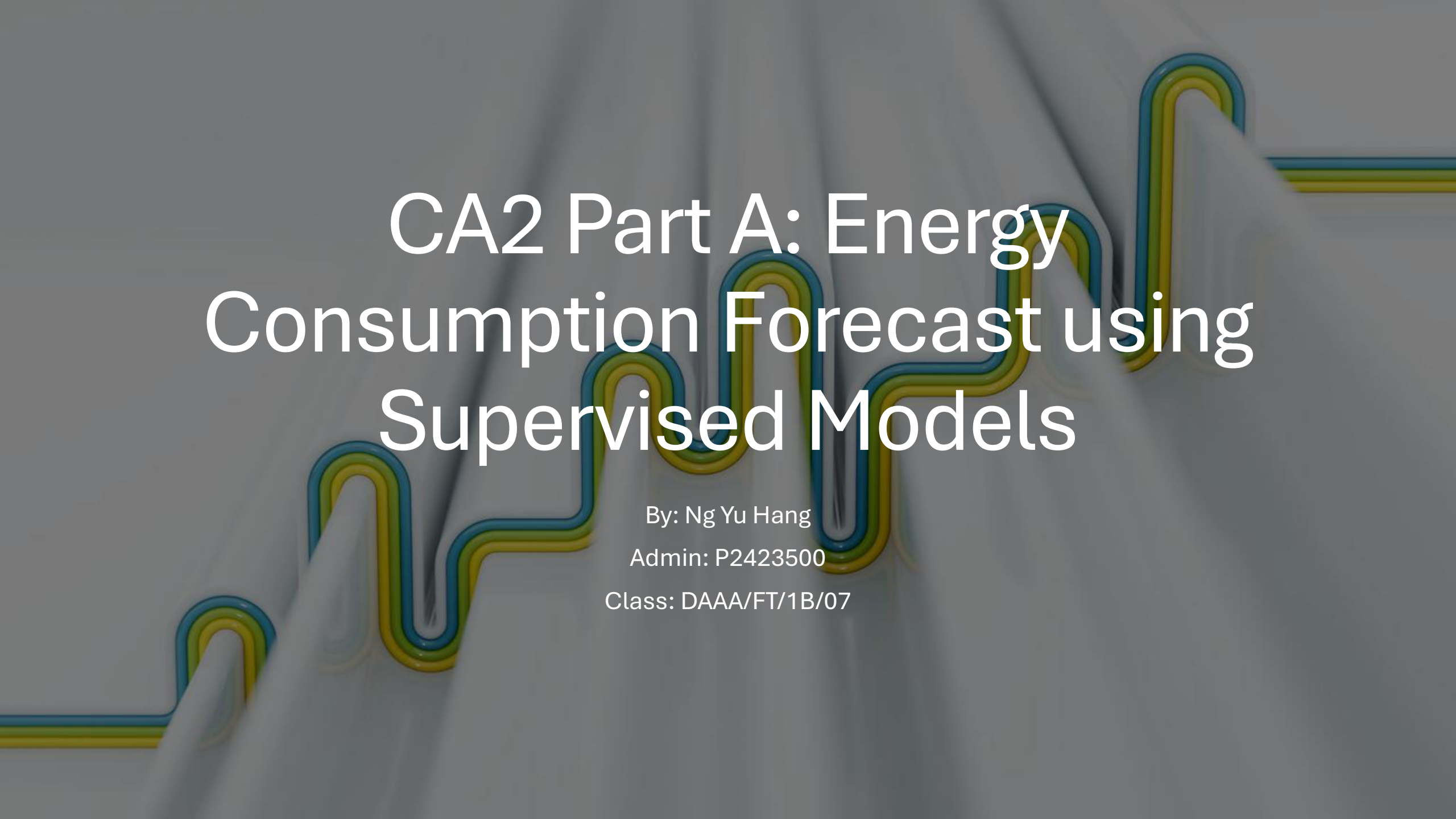




CA2 Part A: Energy Consumption Forecast using Supervised Models

By: Ng Yu Hang

Admin: P2423500

Class: DAAA/FT/1B/07

Project Overview

Background:

Using the provided Energy Consumption Dataset, the task is to train time series models and forecast the consumption of **gas**, **electricity** and **water** for the next 60 months (**5 years**)

Phases of this project

1. Data Preparation – Handling outliers
2. Exploratory Data Analysis – Visualize Data structure and trends
3. Stationary Testing – Testing for non-stationary data
4. Time Series Forecasting models – Train test split and creating a baseline
5. Model Development for Forecasting – Developing the actual models for each energy type
6. Model Evaluation – Evaluating the performance of our models
7. Residual Diagnostics – Checking for residuals to access model reliability
8. Forecasting Future Energy Consumption – Using the models to forecast energy consumption
9. Final Conclusion

Phase 1: Data Preparation

- Convert dataset date column datetime format, then set **dayfirst = true** so it knows that our dates are in **DD/MM/YYYY** format
- **Data.describe()** to view a summary of our data as well as checking for missing values with **data.isnull().sum()**
- Used the **IQR method** to cap our outliers

```
# Load the dataset
data = pd.read_csv('CA2-Energy-Consumption-Data.csv')

# Convert DATE column to datetime format
data['DATE'] = pd.to_datetime(data['DATE'], dayfirst=True)

# Set DATE as the index for time series analysis
data.set_index('DATE', inplace=True)

# Display the first few rows and basic information
print("Dataset Overview:")
print(data.info())
print("\nMissing Values:")
print(data.isnull().sum())

# Display summary statistics
data.describe()
```

```
# Function to detect and cap outliers using the IQR method
def cap_outliers(series):
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap the outliers
    return series.clip(lower=lower_bound, upper=upper_bound)

# Apply outlier treatment for each energy type
for column in data.columns:
    data[column] = cap_outliers(data[column])
```

Phase 1: Data Preparation

- Feature Engineered **Lag Features** and **Rolling statistics** as well as **date based features**
- Handle their missing values.
- Not using them as exogenous variables, but adding them to the data would allow the models to **catch on seasonal patterns** without directly modifying them.

```
# 1. Lag Features (for Electricity Consumption)
data['Electricity_Lag_1'] = data['Electricity Consumption (MWh)'].shift(1)

# 2. Rolling Statistics for Electricity Consumption
data['Electricity_Rolling_Mean_3'] = data['Electricity Consumption (MWh)'].rolling(window=3).mean()

# 3. Date-Based Features for Electricity
data['Electricity_Month'] = data.index.month
data['Electricity_Quarter'] = data.index.quarter
data['Electricity_Year'] = data.index.year

# 4. Lag Features (for Gas Consumption)
data['Gas_Lag_1'] = data['Gas Consumption (tons)'].shift(1)

# 5. Rolling Statistics for Gas Consumption
data['Gas_Rolling_Mean_3'] = data['Gas Consumption (tons)'].rolling(window=3).mean()

# 6. Date-Based Features for Gas
data['Gas_Month'] = data.index.month
data['Gas_Quarter'] = data.index.quarter
data['Gas_Year'] = data.index.year

# 7. Lag Features (for Water Consumption)
data['Water_Lag_1'] = data['Water Consumption (tons)'].shift(1)

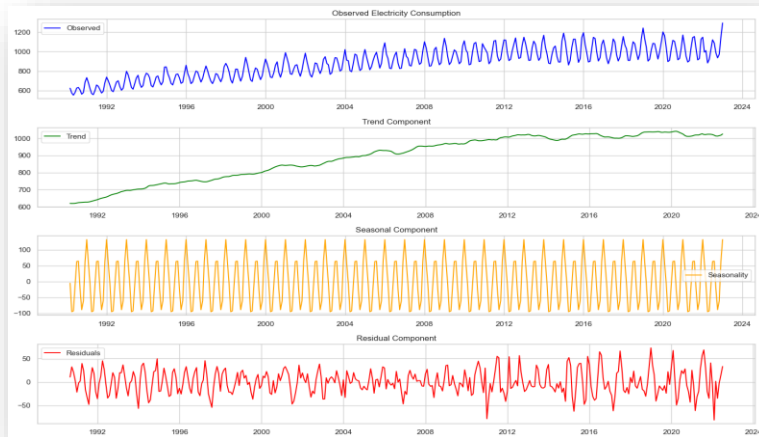
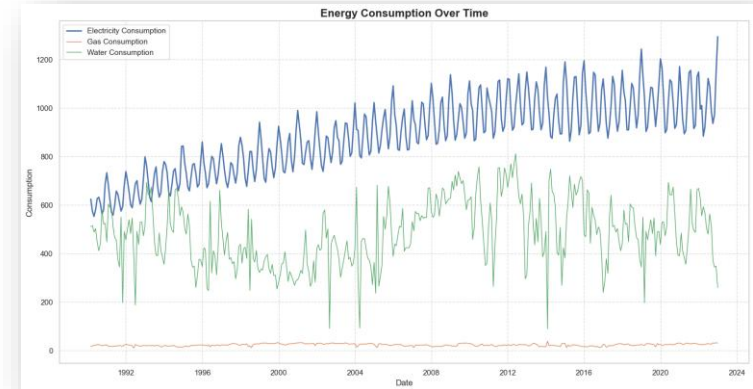
# 8. Rolling Statistics for Water Consumption
data['Water_Rolling_Mean_3'] = data['Water Consumption (tons)'].rolling(window=3).mean()

# 9. Date-Based Features for Water
data['Water_Month'] = data.index.month
data['Water_Quarter'] = data.index.quarter
data['Water_Year'] = data.index.year

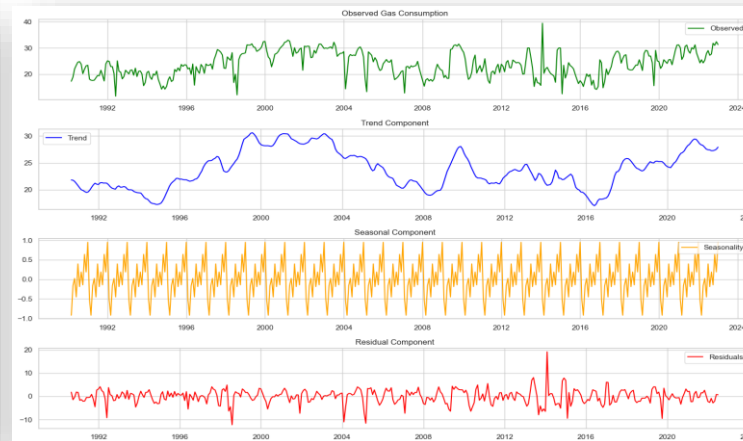
# Handle Missing Values due to Lag/Rolling Features (Drop the rows with NaN values)
data.dropna(inplace=True)
```

Phase 2: Exploratory data analysis

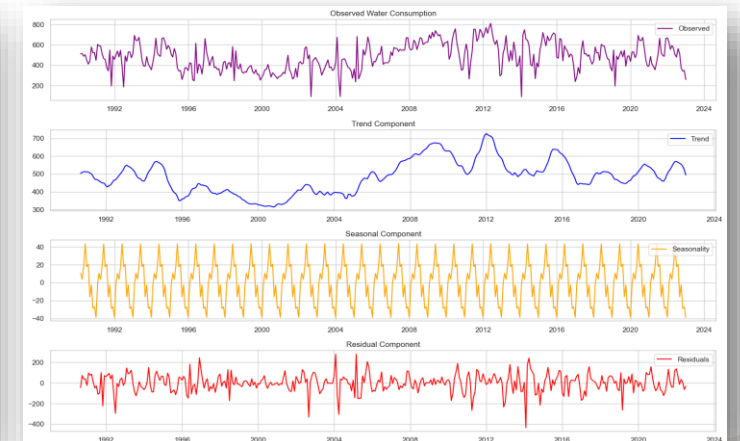
- Plot out all the graphs for each of the energy types (electricity, gas and water)
- Plot out the **seasonal decomposition graph** for all 3 to identify **seasonality** and **trend**



Electricity



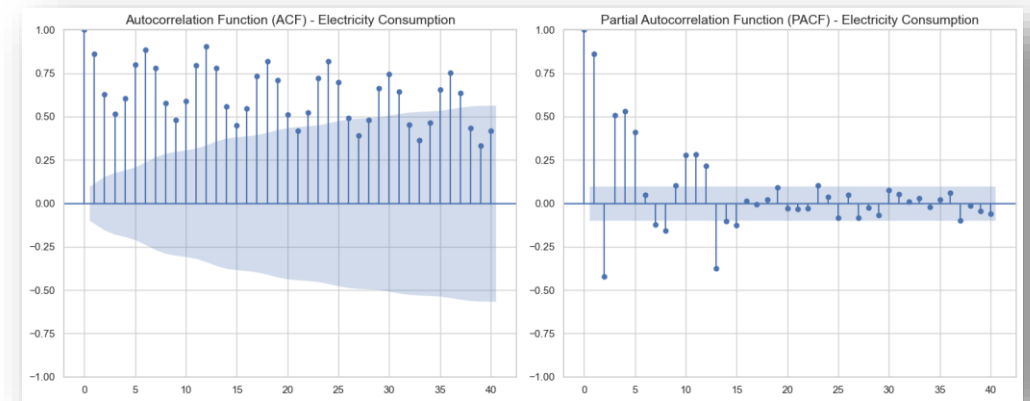
Gas



Water

Phase 2: Exploratory data analysis

- Plot a **ACF** and **PACF** graph for correlation analysis for each energy type
- To view whether there's a relationship between past values with the current value
- ACF – useful for identifying the **Moving average**
- PACF – useful for identifying the **Autoregressive order** of the model



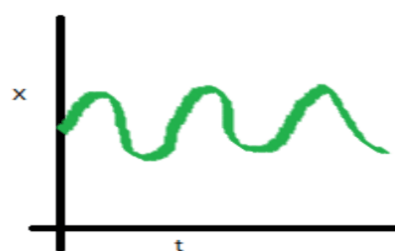
One of the graph plotted (electricity consumption)

We can see that the lags are still extremely high in the ACF graph – seasonal pattern that persists over long periods

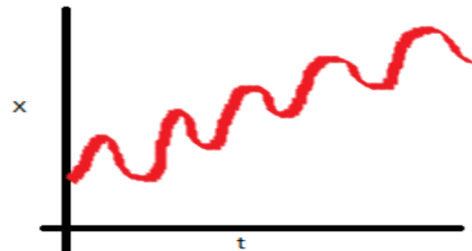
From the PACF graph, we can see a high correlation at lag 1, meaning the best autoregressive component is likely 1 ($p = 1$) – the reason why I fixed $p = 1$ for sarimax model

Phase 3: Stationary Testing

- For the time series models to work, data must be stationary
- Check using ADF (Augmented Dickey-Fuller test, which shows us the P value.
- P value > 0.05 , non stationary
- P value ≤ 0.05 , stationary
- Models need stationary data to predict well.



Stationary series



Non-Stationary series

	Series	ADF Statistic	p-value
0	Electricity Consumption	-2.312505	0.167967
1	Gas Consumption	-3.679822	0.004406
2	Water Consumption	-4.636033	0.000111

$0.167 > 0.05$

$0.0044 < 0.05$

$0.00011 < 0.05$

$d = 1$ is needed in the ARIMA model for electricity consumption

Why? No need to inverse difference back, as the model would handle it by itself if set $d = 1$

Phase 4: Time series forecasting models

- Decided that the models we use for each energy type

Energy Type	Model	Why?
Electricity	SARIMAX	Strong seasonality + trend
Gas	SARIMAX	Moderate seasonality + volatile data
Water	Holts Winter	Slightly weaker seasonality + fluctuations in residuals

- Train Test Split to prepare for the training and testing of models

```
# Train-Test Split (80% Train, 20% Test)
train_size = int(len(data) * 0.8)

# Splitting the entire dataset
train_data = data.iloc[:train_size]
test_data = data.iloc[train_size:]

# Splitting data for each consumption type
train_electricity = train_data['Electricity Consumption (MWh)']
test_electricity = test_data['Electricity Consumption (MWh)']

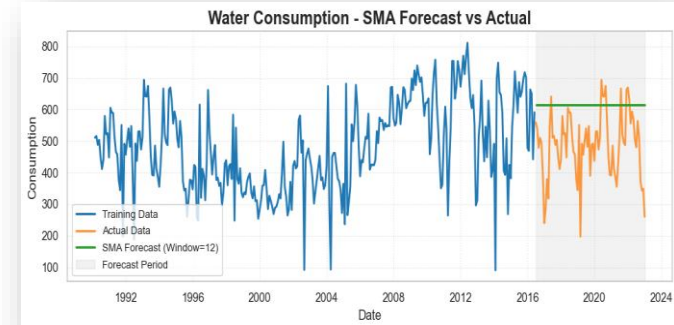
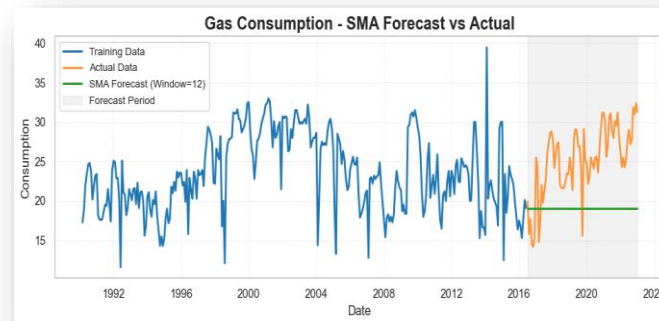
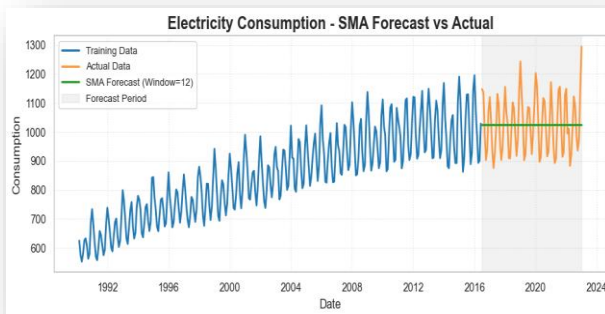
train_gas = train_data['Gas Consumption (tons)']
test_gas = test_data['Gas Consumption (tons)']

train_water = train_data['Water Consumption (tons)']
test_water = test_data['Water Consumption (tons)']
```

Phase 4: Time series forecasting models

- Trained a baseline using SMA (Simple Moving Average) – Not a model but useful for baseline
- Other models expected to outperform this
- SMA is useful because – it's a simple model that just takes the average of the last few observations and instead smooths identifying the long term trend.

	Energy Type	MSE	RMSE	MAE	R ²	MAPE (%)
0	Electricity Consumption	9267.701828	96.268904	81.193354	-0.000475	7.919438
1	Gas Consumption	56.441124	7.512731	6.799051	-1.938621	25.892573
2	Water Consumption	26706.397746	163.420922	136.722574	-1.540457	34.718249



Phase 5: Model development for forecasting

- SARIMAX for Electricity and Gas Consumption
- HWES for Water Consumption
- Calculating P value again with ADF test on training set (more reliable, as testing data will not be seen by the models, provides a more unbiased evaluation)

ADF Test Results for Electricity (Training Data):

ADF Statistic: -1.8507392959671922
p-value: 0.35557758001046086

$0.355 > 0.05$

ADF Test Results for Gas (Training Data):

ADF Statistic: -3.398377396823933
p-value: 0.011014310562745778

$0.011 < 0.05$

ADF Test Results for Water (Training Data):

ADF Statistic: -3.3770481281144393
p-value: 0.01176663132688018

$0.012 < 0.05$

Phase 5: Model development for forecasting

- SARIMAX models used manual grid search to test for parameters that yields the best performance metrics.
- The output will show the best parameters for the energy type, which we will use to forecast our data at the end.
- Fixed $p = 1$ as seen from the PACF plot
- Fixed $d = 1$ for electricity as electricity requires differencing

```
# Step 2: Define Function to Perform Grid Search
def sarimax_grid_search_electricity(train, target):
    best_mape_electricity = float('inf')
    best_aic_electricity = float('inf')
    best_order_electricity = None
    best_seasonal_order_electricity = None
    best_forecast_electricity = None
    best_model_electricity = None

    for param in pdq_electricity:
        for seasonal_param in seasonal_pdq_electricity:
            try:
                # Fit SARIMAX Model
                model = SARIMAX(train[target],
                                order=param,
                                seasonal_order=seasonal_param,
                                enforce_stationarity=False,
                                enforce_invertibility=False)

                result = model.fit(disp=False)
                forecast = result.forecast(steps=len(test_data)) # Using the test data length
                forecast.index = test_data.index # Align forecast with test data

                # Performance Metrics (MAPE and AIC)
                mape = mean_absolute_percentage_error(test_data[target], forecast)
                aic = result.aic

                # Model Selection Criteria
                if mape < best_mape_electricity or (mape == best_mape_electricity and aic < best_aic_electricity):
                    best_mape_electricity = mape
                    best_aic_electricity = aic
                    best_order_electricity = param
                    best_seasonal_order_electricity = seasonal_param
                    best_forecast_electricity = forecast
                    best_model_electricity = result

            except Exception as e:
                continue
```

	Energy Type	Order	Seasonal_Order	MAPE	MSE	RMSE	MAE	R2_Score	AIC
0	Electricity Consumption (MWh)	(1, 1, 1)	(1, 0, 0, 12)	0.03565	1953.283587	44.195968	36.34182	0.789137	2839.604826

Phase 5: Model development for forecasting

Why SARIMAX and not SARIMA?

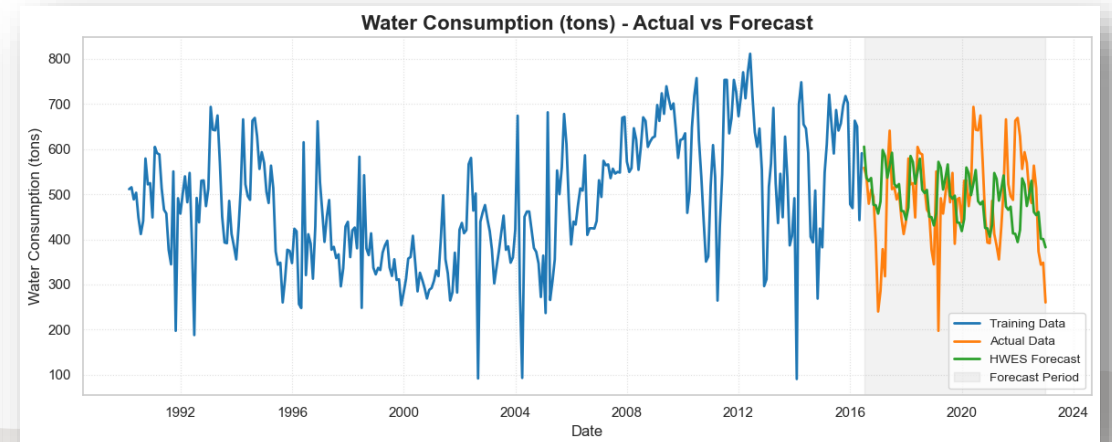
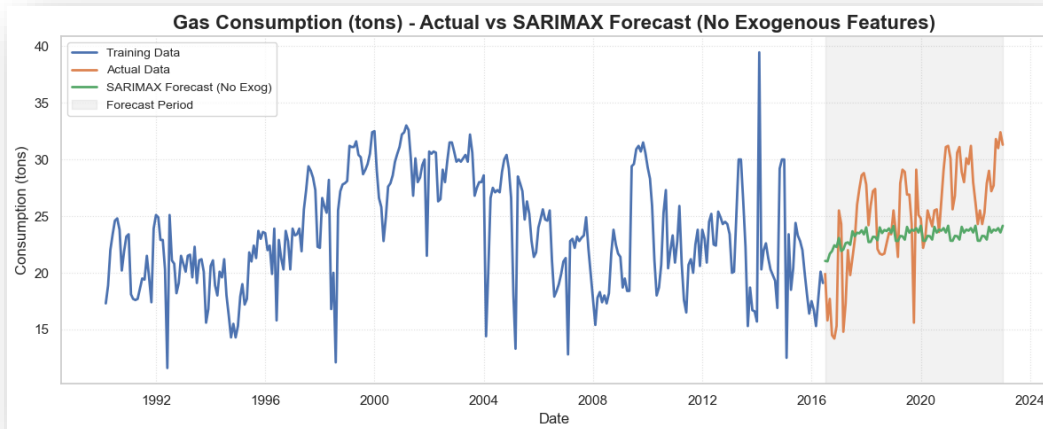
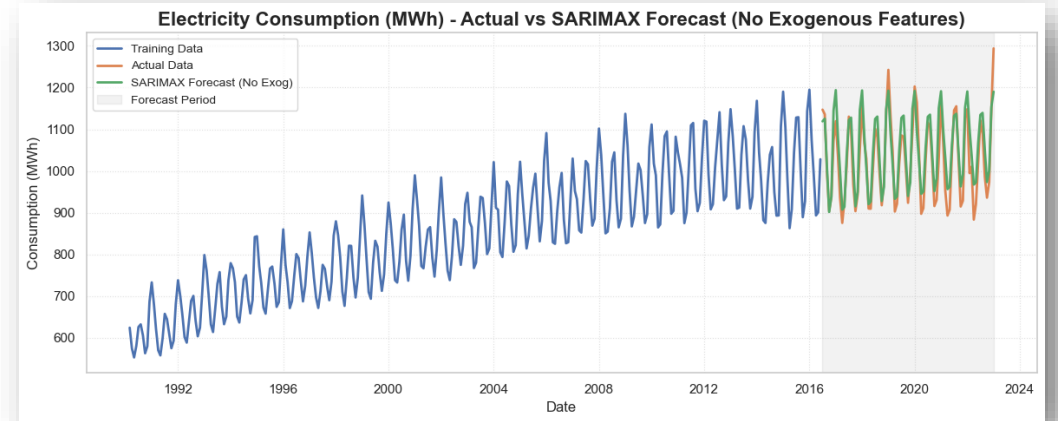
- SARIMAX is an extension of SARIMA. SARIMAX is able to take into account of external factors (though not provided, when provided, it can be used in this for project to create excellent models.) Hence, the reason for SARIMAX instead of SARIMA, is for the efficiency for reuse for future projects.

Why HWES and not SARIMAX for water consumption?

- Because from the seasonal decomposition graph, we can see that there are irregular fluctuations which makes HWES much better at capturing it by smoothing it out, as they are more robust to the fluctuations. SARIMAX may over complex it due to its complex nature and easily overfit/underfit while HWES focuses on general trend and seasonality.

Phase 5: Model development for forecasting

- All model predictions are plotted out against the testing set, for visualization of how accurate it is, without just viewing at the scores.

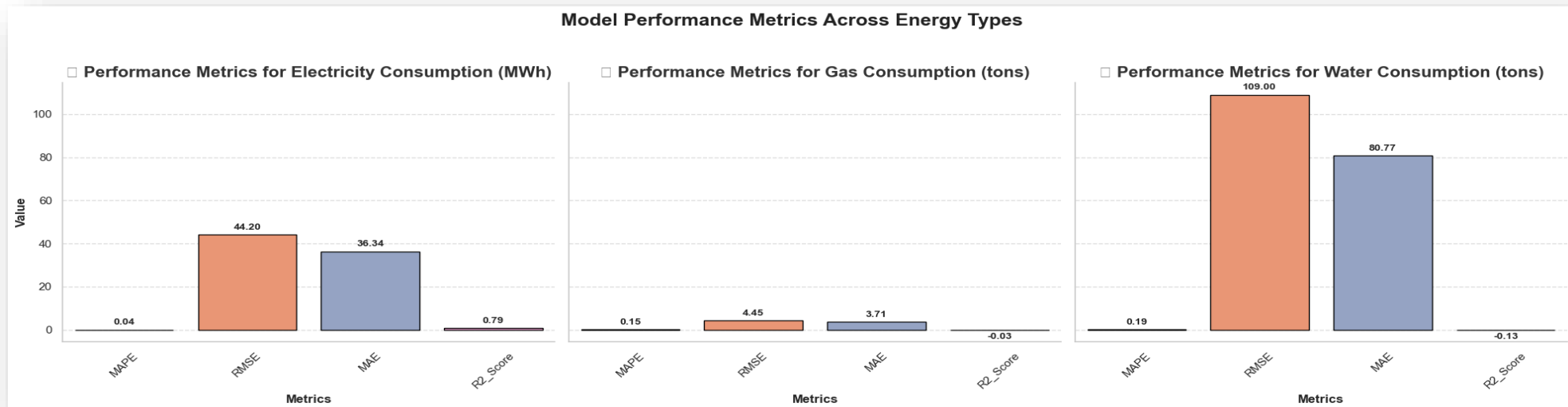


Phase 6: Model Evaluation

- Compile the model performances of each of the energy types
- Plot out a graph to see their general performances of their respective models.

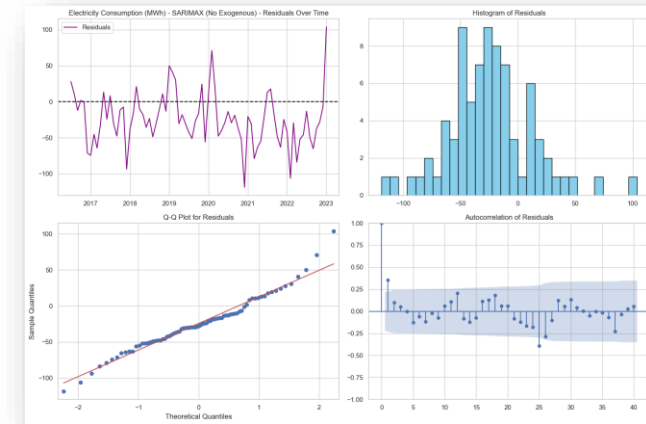
Model Performance Summary:

Energy Type	Model	MAPE (%)	MSE	RMSE	MAE	R ² Score	AIC
Electricity Consumption	SARIMAX (Electricity)	3.57%	1953.28	44.20	36.34	0.79	2839.60
Gas Consumption	SARIMAX (Gas)	15.24%	19.79	4.45	3.71	-0.03	1591.75
Water Consumption	HWES (Water)	19.12%	11881.32	109.00	80.77	-0.13	2992.09

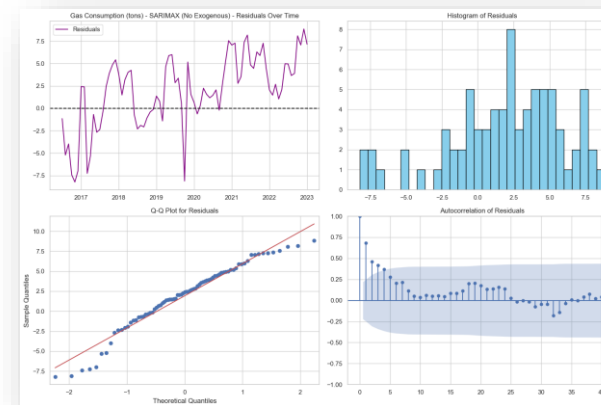


Phase 7: Residual Diagnostics to best models

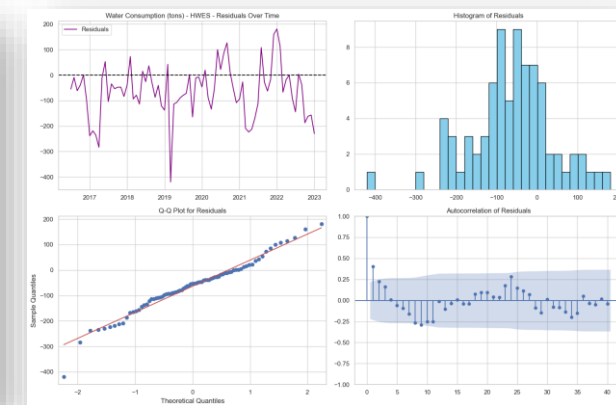
- Residual tests are to help check if the models are reliable.
- Residuals = predicted values – actual values
- What we want – the values are random and unpredictable
- If the errors are showing a pattern – its unable to capture important trends/cycles
- Plotted Residual plots.
- **Electricity and water** generally have normal residuals, however water has more fluctuations meaning it couldn't capture extreme events.
- An improvement could be using SARIMAX model if exogenous variables are provided such as pricing, rainfall, population.



Electricity



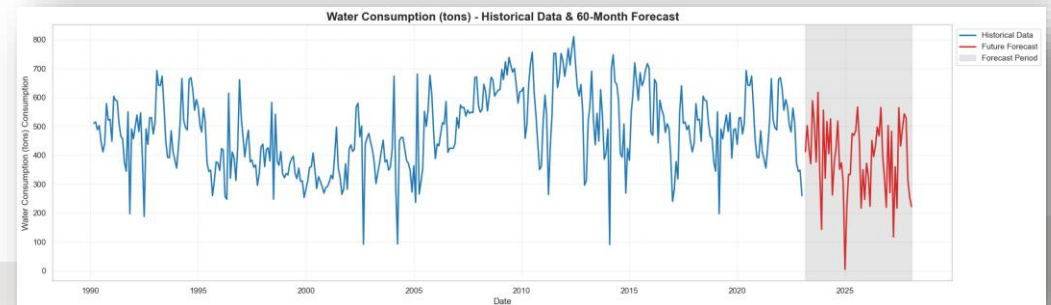
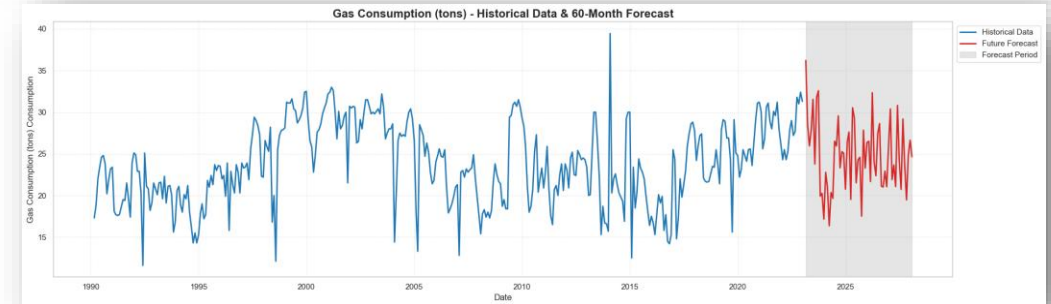
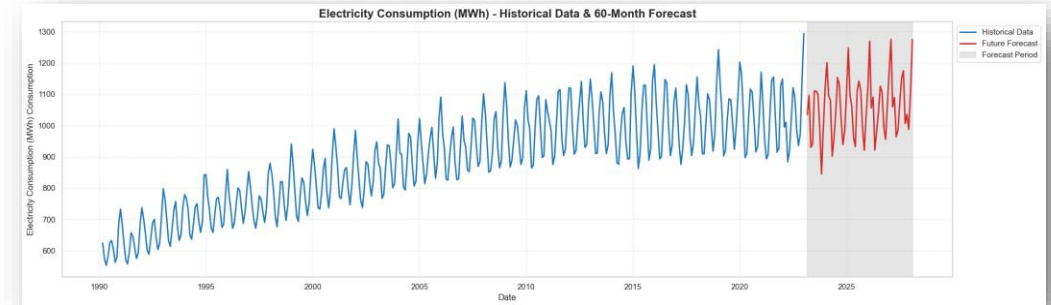
Gas



Water

Phase 8: Forecasting future energy consumption

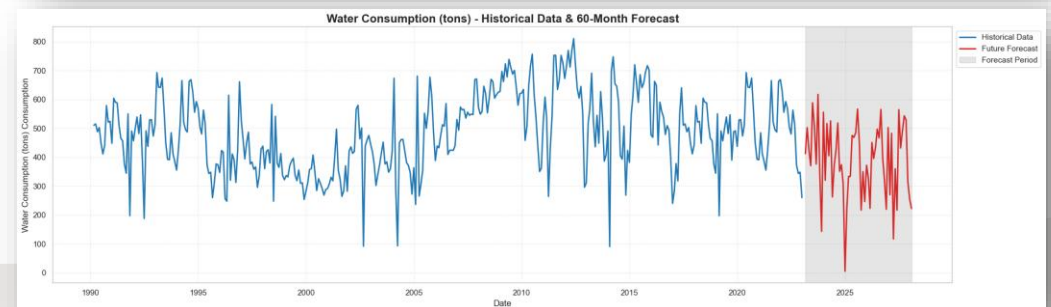
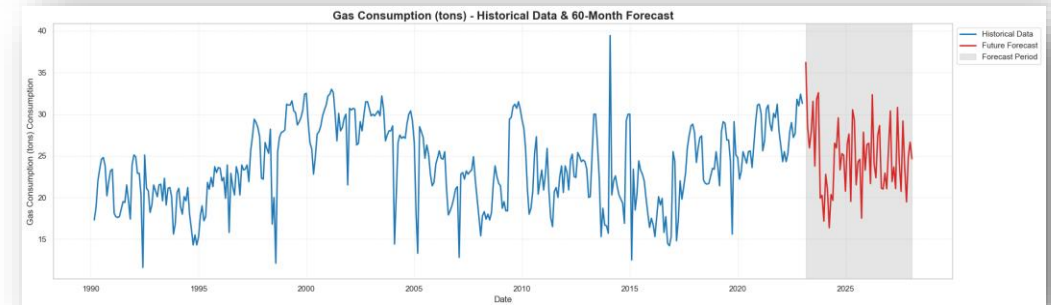
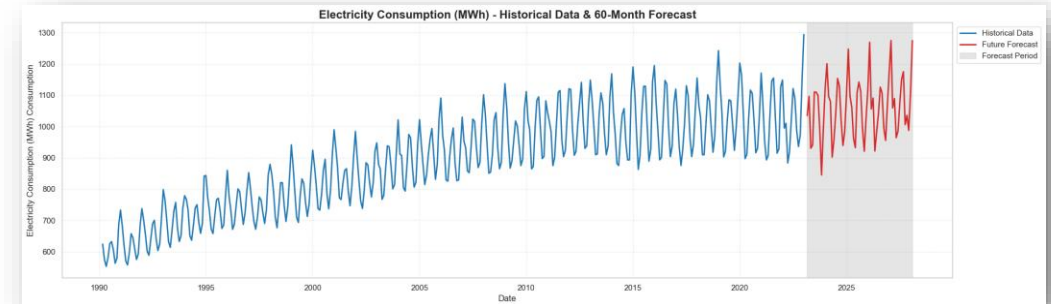
- Now using our best models, we will try to forecast the energy consumption for the next **60 months**.
- Train the best model on the entire dataset to forecast.
- Added **variance noise** derived from the **residuals** of each model to the forecasted values.
- By calculating the spread of the residuals, we can apply it to the forecasted values to imitate real world forecast data.



Phase 9: Final Conclusion

- Based on our final forecasts, we can conclude that our models, even though not perfect, are able to capture general trends and provide realistic forecasts for the next 60 months.
- The following configurations are our best models:

Energy Type	Model	Order	Seasonal Order	Trend	Seasonal	Damped
Electricity Consumption (MWh)	SARIMAX	(1, 1, 1)	(1, 0, 0, 12)	NaN	NaN	NaN
Gas Consumption (tons)	SARIMAX	(1, 0, 1)	(0, 1, 1, 12)	NaN	NaN	NaN
Water Consumption (tons)	HWES	NaN	NaN	mul	add	False



Suggestions for improvement

- Due to computational and time constraints, the models are trained with only ranges 0 to 1. With the use of higher ranges, better models can be created that create more accurate and reliable forecasts
- As our models use SARIMAX, it can be easily reused to implement the exogenous variables, along with our feature engineered variables to produce better models.
- For water consumption, we could opt for SARIMAX when exogenous variables are provided, which can create a more fine tuned model.